Question assigned to the following page: 1

CS771: Introduction to Machine Learning

Minor Assignment 2 — Report

Generative Models for Classification and Image Inpainting with Missing Pixels

Suvid Goel, Roll: 231065
Department of Computer Science and Engineering, IIT Kanpur
gsuvid23@iitk.ac.in

1 Part 1: Single-Step Joint Inference — Derivation (15 marks)

1.1 Setup and Notation

Each class $c \in \{0, \ldots, C-1\}$ is modelled by a single Gaussian:

$$P[x \mid y = c] = \mathcal{N}(x;\, \mu^c,\, \Sigma^c).$$

A test image x is partitioned into observed pixels x^o and missing pixels x^m. Conformably partitioning the class parameters:

$$\mu^c = \begin{bmatrix} \mu^{o,c} \\ \mu^{m,c} \end{bmatrix}, \qquad \Sigma^c = \begin{bmatrix} \Sigma^{oo,c} & \Sigma^{om,c} \\ \Sigma^{mo,c} & \Sigma^{mm,c} \end{bmatrix}. \tag{1}$$

Here d_o and d_m denote the number of observed and missing dimensions.

1.2 Lecture Code: Two-Step Inference

The lecture code performs inference sequentially:

$$\textbf{Step 1:} \quad \hat{y} = \arg\max_c \; P[y = c \mid x^o] = \arg\max_c \; P[x^o \mid y = c] \cdot P[y = c], \tag{2}$$

$$\textbf{Step 2:} \quad \hat{x}^m = \arg\max_v \; P[v \mid x^o,\, y = \hat{y}]. \tag{3}$$

By the standard Gaussian conditional formula, $P[x^m \mid x^o, y = c] = \mathcal{N}(\bar{\mu}^c, \bar{\Sigma}^c)$, where

$$\bar{\mu}^c = \mu^{m,c} + \Sigma^{mo,c}(\Sigma^{oo,c})^{-1}(x^o - \mu^{o,c}), \tag{4}$$

$$\bar{\Sigma}^c = \Sigma^{mm,c} - \Sigma^{mo,c}(\Sigma^{oo,c})^{-1}\Sigma^{om,c}. \tag{5}$$

Since the mode of a Gaussian equals its mean, $\hat{x}^m = \bar{\mu}^{\hat{y}}$.

1.3 Single-Step Joint Inference

We solve the joint optimisation:

$$\{\hat{y},\, \hat{x}^m\} = \arg\max_{c,\, v} \; P[y = c,\; x^m = v \mid x^o]. \tag{6}$$

Applying the product rule. Dropping $P[x^o]$ (constant w.r.t. c and v) via Bayes' rule:

$$\begin{aligned} P[y = c,\, x^m = v \mid x^o] \;&\propto\; P[x^o,\, x^m = v \mid y = c] \cdot P[y = c] \\ &= P[x^m = v \mid x^o,\, y = c] \cdot P[x^o \mid y = c] \cdot P[y = c]. \end{aligned} \tag{7}$$

Inner maximisation over v (fixed c). Because $P[x^m = v \mid x^o, y = c] = \mathcal{N}(v; \bar{\mu}^c, \bar{\Sigma}^c)$, its mode is $v_c^* = \bar{\mu}^c$, since mode of Gaussian is its mean, with maximum value $(2\pi)^{-d_m/2}|\bar{\Sigma}^c|^{-1/2}$.
Substituting back:

$$\max_v \; P[y = c,\, x^m = v \mid x^o] \;\propto\; P[x^o \mid y = c] \cdot P[y = c] \cdot |\bar{\Sigma}^c|^{-1/2}. \tag{8}$$

Preprint.

Question assigned to the following page: 1

Outer maximisation over c. Taking logs of (8) and dropping the constant $-\frac{d_m}{2}\log(2\pi)$:

$$\hat{y} = \arg\max_c \left[\underbrace{\log P[x^o \mid y=c] + \log P[y=c]}_{\text{two-step score (unchanged)}} - \underbrace{\tfrac{1}{2}\log|\bar{\Sigma}^c|}_{\text{new penalty}} \right] \tag{9}$$

Expanding $\log P[x^o \mid y=c]$ using the Gaussian log-likelihood:

$$\log P[x^o \mid y=c] = -\frac{d_o}{2}\log(2\pi) - \frac{1}{2}\log|\Sigma^{oo,c}| - \frac{1}{2}(x^o-\mu^{o,c})^\top(\Sigma^{oo,c})^{-1}(x^o-\mu^{o,c}). \tag{10}$$

Dropping the constant $-\frac{d_o}{2}\log(2\pi)$ (independent of c), the full single-step score for class c becomes:

$$\text{score}(c) = \underbrace{-\frac{1}{2}\log|\Sigma^{oo,c}| - \frac{1}{2}(x^o-\mu^{o,c})^\top(\Sigma^{oo,c})^{-1}(x^o-\mu^{o,c}) + \log P[y=c]}_{\text{two-step score}} - \underbrace{\frac{1}{2}\log|\bar{\Sigma}^c|}_{\text{new penalty}}. \tag{11}$$

The predicted class and missing pixels are therefore:

$$\hat{y} = \arg\max_c \ \text{score}(c), \tag{12}$$

$$\hat{x}^m = \bar{\mu}^{\hat{y}} = \mu^{m,\hat{y}} + \Sigma^{mo,\hat{y}}(\Sigma^{oo,\hat{y}})^{-1}(x^o-\mu^{o,\hat{y}}). \tag{13}$$

Both quantities fall out of a single pass — $\bar{\mu}^c$ is computed for every class during scoring (inner max over v), and $\hat{y}$ selects which one to use (outer max over c).

1.4 Are the Two Methods Identical?

No. The two-step score for class c is $\log P[x^o \mid y=c] + \log P[y=c]$. The single-step score adds $-\frac{1}{2}\log|\bar{\Sigma}^c|$, which depends on c and measures the log-volume of the conditional covariance of the missing pixels. The single-step method:

- *penalises* classes for which the missing-pixel conditional distribution is highly diffuse (large $|\bar{\Sigma}^c|$);
- *favours* classes that predict the missing region confidently.

The two methods agree only when $|\bar{\Sigma}^c|$ is identical for every class — which is not the case for real-world data such as MNIST.

Question assigned to the following page: 2

2 Part 2: Code Modifications (15 marks)

2.1 Design Decisions

Matrix inverse. The lecture code uses `numpy.linalg.pinv` (Moore–Penrose pseudo-inverse) to handle rank-deficient covariance sub-blocks (pixels that are always zero in a class have zero variance). We follow this convention for $(\Sigma^{oo,c})^{-1}$ and use `lin.slogdet` for $\log|\bar{\Sigma}^c|$, setting the log-determinant to zero when `slogdet` returns a non-positive sign — identical to the lecture code's handling of degenerate covariances.

Function merging. Because single-step inference needs both the class scores and the conditional means $\bar{\mu}^c$ in one pass, we replace `predictGen` with a new function `predictSingleStep`. The original `predictClassScores` is called unchanged inside this function. `reconstructImages` from the lecture code requires no modification — a separate helper `reconstructFromXm` handles the trivial assembly of the full image from $\hat{x}^m$ and the observed pixels, which is identical bookkeeping for both methods.

2.2 Pseudocode

Algorithm 1 PREDICTSINGLESTEP$(X, \text{model}, C, \texttt{mask})$

Require: Test matrix $X \in \mathbb{R}^{n\times d}$, trained model, boolean array `obs` (`True` = observed)
Ensure: Predicted labels $\hat{y} \in \{0,\dots,C-1\}^n$, missing pixel estimates $\hat{X}^m \in \mathbb{R}^{n\times d_m}$

`mis` $\leftarrow \neg$`obs`; `classScores` $\leftarrow \mathbf{0}_{n\times C}$; `all_xm` $\leftarrow \mathbf{0}_{n\times C\times d_m}$
for $(\mu^c, \Sigma^c, c, p^c) \in$ model **do**
  $S_{oo} \leftarrow \Sigma^c[\texttt{obs}, \texttt{obs}]$; $S_{mo} \leftarrow \Sigma^c[\texttt{mis}, \texttt{obs}]$; $S_{mm} \leftarrow \Sigma^c[\texttt{mis}, \texttt{mis}]$
  $A^c \leftarrow S_{mo} \cdot \texttt{pinv}(S_{oo})$ ▷ $(d_m \times d_o)$, computed once, used twice below
  $\bar{\Sigma}^c \leftarrow S_{mm} - A^c S_{mo}^\top$ ▷ Schur complement
  $(\text{sgn}, \ell) \leftarrow \texttt{slogdet}(\Sigma^c)$; $\ell_c \leftarrow \ell$ **if** sgn > 0 **else** 0
  `classScores`$[:, c] \leftarrow$ `predictClassScores`$(X, \mu^c, \Sigma^c, p^c, \texttt{obs}) - \frac{1}{2}\ell_c$
  `all_xm`$[:, c, :] \leftarrow \mu^c[\texttt{mis}] + (X[:, \texttt{obs}] - \mu^c[\texttt{obs}]) \cdot A^{c\top}$ ▷ $\bar{\mu}^c$ for all test points
end for
$\hat{y} \leftarrow$ `argmax`(`classScores`, axis $= 1$)
$\hat{X}^m \leftarrow$ `all_xm`$[\texttt{arange}(n), \hat{y}, :]$ ▷ pick $\bar{\mu}^c$ for winning class
return $\hat{y}, \hat{X}^m$

Question assigned to the following page: 2

2.3 Python Code

```
def predictSingleStep(X, model, C, mask):
    """
    Solves: {y_hat, x_m_hat} = argmax_{c,v} P[y=c, x^m=v | x^o]

    For each fixed c, the optimal v = mu_bar^c (conditional mean) is
    known analytically. We precompute mu_bar^c for ALL classes during
    the scoring loop, then pick the one for the winning class after
    argmax --- matching the inner-then-outer maximisation in the derivation.

    Single-step score for class c:
        log P[x^o|y=c] + log P[y=c]  -  0.5 * log|Sigma_bar^c|
    where Sigma_bar^c = Sigma^{mm,c} - A^c Sigma^{om,c}
          A^c         = Sigma^{mo,c} (Sigma^{oo,c})^{-1}
    """
    obs, mis = mask, ~mask
    n, d_m   = X.shape[0], mis.sum()

    classScores = np.zeros((n, C))
    all_xm      = np.zeros((n, C, d_m))

    for mu, Sigma, c, p in model:
        Soo = Sigma[np.ix_(obs, obs)]
        Smo = Sigma[np.ix_(mis, obs)]
        Smm = Sigma[np.ix_(mis, mis)]

        # A^c computed once, reused for both Schur complement and mu_bar^c
        A = Smo @ lin.pinv(Soo)

        # Schur complement = conditional covariance of x^m given x^o
        Sigma_bar     = Smm - A @ Smo.T
        sign, logdet = lin.slogdet(Sigma_bar)
        logdet_c      = logdet if sign > 0 else 0.0

        # Single-step score: two-step score minus 0.5 * log|Sigma_bar^c|
        classScores[:, c] = (predictClassScores(X, mu, Sigma, p, obs)
                             - 0.5 * logdet_c)

        # Precompute mu_bar^c for all test points simultaneously
        all_xm[:, c, :] = mu[mis] + (X[:, obs] - mu[obs]) @ A.T

    # Maximise over c to get y_hat
    yPred = np.argmax(classScores, axis=1)

    # Pick precomputed mu_bar^c for the winning class
    xm_hat = all_xm[np.arange(n), yPred, :]

    return yPred, xm_hat
```

Question assigned to the following page: 2

```python
def reconstructFromXm(X, xm_hat, mask):
    """
    Assemble full image from observed pixels + x_m_hat.
    Pure bookkeeping --- identical for both methods.
    """
    XRecon = X.copy()
    XRecon[:, ~mask] = xm_hat
    return truncatePixels(XRecon)
```

Key points:

- `predictSingleStep` replaces `predictGen` and returns *both* $\hat{y}$ and $\hat{x}^m$ — the two outputs of the joint optimisation defined in equation (6). `predictClassScores` is called unchanged inside the loop.
- $A^c = \Sigma^{mo,c}(\Sigma^{oo,c})^+$ is computed *once* per class and used for both the Schur complement $\bar{\Sigma}^c = \Sigma^{mm,c} - A^c S_{mo}^\top$ and the conditional mean $\bar{\mu}^c = \mu^{m,c} + A^c(x^o - \mu^{o,c})$, avoiding a redundant `pinv` call.
- $\bar{\mu}^c$ is precomputed for *all* classes during the scoring loop and stored in `all_xm[:,c,:]`, directly matching the inner-then-outer maximisation structure of the derivation: inner max over v (precomputing $\bar{\mu}^c$) happens inside the loop, outer max over c (argmax) happens after.
- `reconstructFromXm` assembles the full image from $\hat{x}^m$ and the observed pixels — identical bookkeeping for both methods. `reconstructImages` from the lecture code requires **no modification**.
- `lin.pinv` and `lin.slogdet` are used throughout, consistent with the lecture code's robustness conventions.

Question assigned to the following page: 3

3 Part 3: Experimental Comparison (20 marks)

3.1 Experimental Setup

We train one Gaussian per class ($C = 10$) on all 60 000 MNIST training images using the unmodified MLE from the lecture code. Evaluation uses all 10 000 test images under five censoring configurations from the assignment:

Censoring config	Missing pixels	Label
`X[:,4:24,4:24]=0`	$\approx 51\%$	aggressive
`X[:,6:22,6:22]=0`	$\approx 32\%$	moderate-high
`X[:,8:20,8:20]=0`	$\approx 18\%$	moderate (lecture default)
`X[:,10:18,10:18]=0`	$\approx 8\%$	mild
`X[:,12:16,12:16]=0`	$\approx 2\%$	minimal

3.2 Classification Accuracy

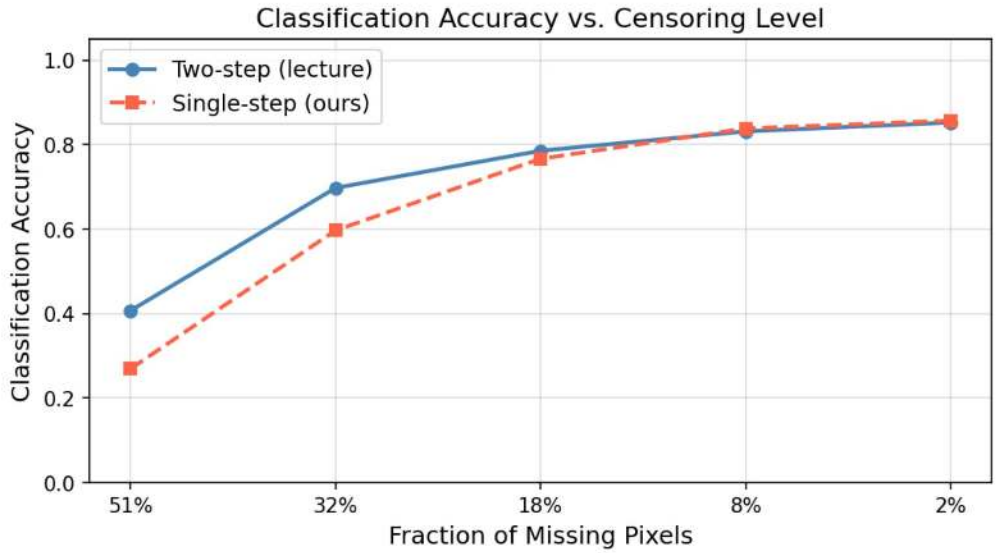


Figure 1: Classification accuracy of two-step (lecture) and single-step (ours) methods across five censoring levels.

Figure 1 shows the classification accuracy of both methods.

- **Both methods improve monotonically as censoring decreases.** Fewer missing pixels means more discriminative information is retained in x^o, benefiting both classifiers.
- **The two-step (lecture) method outperforms single-step at aggressive censoring (51%, 32%)** - At 51% missing pixels, two-step achieves ≈0.42 accuracy vs. ≈0.27 for single-step — a substantial gap. It has a clear numerical explanation. At high censoring, d_o is small (very few observed pixels), making $\Sigma^{oo,c}$ a small, often near-singular submatrix. The pseudo-inverse $(\Sigma^{oo,c})^+$ is a poor approximation in this regime, causing the Schur complement $\bar{\Sigma}^c = \Sigma^{mm,c} - \Sigma^{mo,c}(\Sigma^{oo,c})^+\Sigma^{om,c}$ to be numerically unreliable. The resulting log-determinant $\log|\bar{\Sigma}^c|$ is therefore inaccurate, and since d_m is large, this term is also large in magnitude — it dominates the single-step score and overrides the more reliable observed-pixel likelihood, causing incorrect class selections. The two-step method, which simply ignores this noisy penalty term, is more robust here.

Question assigned to the following page: 3

- **Both methods nearly converge at mild censoring (8%, 2%).** With few missing pixels, $\Sigma^{oo,c}$ is a large, well-conditioned matrix and the Schur complement is computed accurately. Furthermore, $\log|\bar{\Sigma}^c|$ becomes nearly equal across classes (since x^m is a small uninformative sub-vector), so the penalty contributes negligible extra discrimination and both methods produce near-identical scores.

3.3 Inference Time

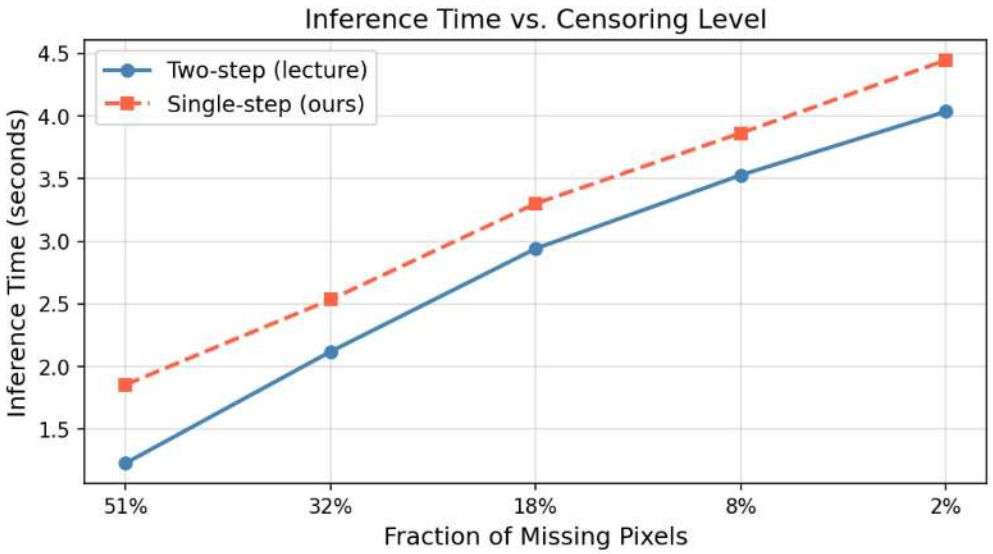


Figure 2: Wall-clock inference time on 10 000 MNIST test images.

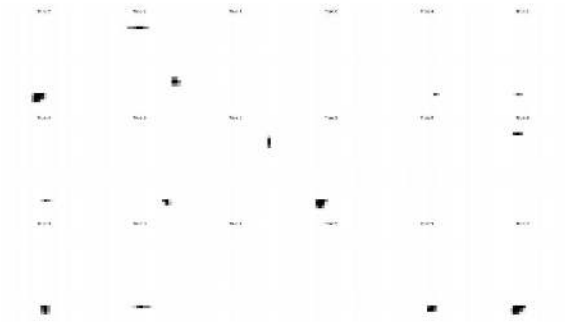
Figure 2 shows inference times across censoring levels, with both methods timed end-to-end including reconstruction. Specifically, two-step timing covers both `predictGen` (Step 1) and `reconstructImages` (Step 2), while single-step timing covers `predictSingleStep` which outputs both $\hat{y}$ and $\hat{x}^m$ in a single pass.

- **Both methods slow down at nearly the same rate as censoring decreases.** The dominant cost for both is `predictClassScores`, which computes $(X^o - \mu^{o,c})(\Sigma^{oo,c})^+$ for all n test points — an $O(n\, d_o^2)$ operation per class. As d_o grows from ≈384 (51%) to ≈768 (2%), this term scales roughly quadratically, explaining why both curves rise steeply and in parallel from left to right.
- **Single-step is consistently slightly slower at every censoring level, with a nearly constant gap.** The extra cost in single-step comes from two sources: the Schur complement $\bar{\Sigma}^c = \Sigma^{mm,c} - A^c S_{mo}^\top$, costing $O(C\, d_m^2\, d_o)$, and `slogdet`$(\bar{\Sigma}^c)$, costing $O(C\, d_m^3)$, both computed once per class. Since these are computed once per class (not per test point), their absolute cost is small compared to the shared $O(n\, d_o^2)$ bottleneck, keeping the gap small and nearly constant across all censoring levels.
- **The Schur complement overhead decreases as censoring decreases.** As d_m shrinks from ≈400 (51%) to ≈16 (2%), the Schur complement cost $O(C\, d_m^2\, d_o)$ drops from ≈61M to ≈0.2M operations. This partially offsets the growing $O(n\, d_o^2)$ cost, which is why the gap between the two curves remains nearly constant rather than growing — the single-step overhead actually becomes cheaper as censoring decreases.

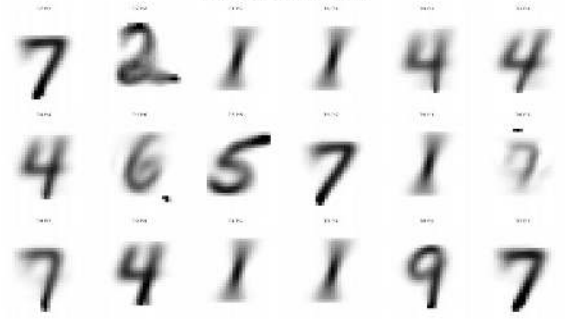
Question assigned to the following page: 3

3.4 Reconstruction Quality

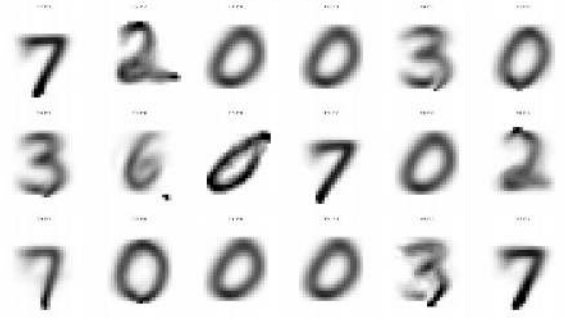
Both methods use the *identical* fill formula (equation (4)); they differ only in which class label drives the reconstruction. Reconstruction quality is therefore directly coupled to classification accuracy. We show censored images and reconstructions for all five censoring levels below.



(a) Censored (51%)



(b) Two-step

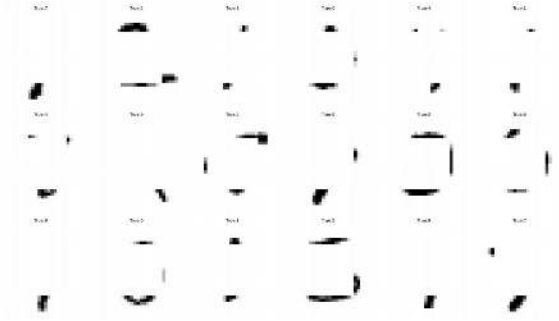


(c) Single-step

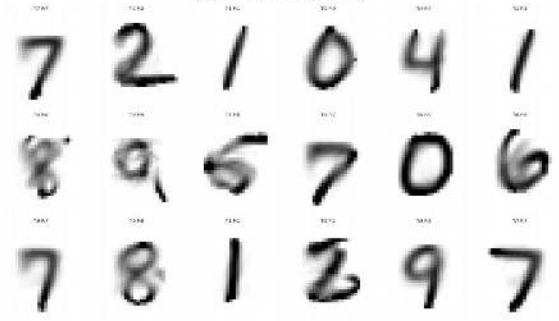
Figure 3: Reconstructions at 51% missing pixels (aggressive censoring).

High censoring (51%, Figure 3). At 51% missing pixels the censored images are almost entirely blank — only scattered border dots remain, making the digit class virtually unidentifiable from x^o alone. The two-step reconstructions are visually diverse and broadly plausible, reflecting the variety of true labels in the sample. The single-step reconstructions, by contrast, collapse almost entirely to the digit **0**: nearly every test image is assigned to class 0 regardless of its true label. This is a direct consequence of the Schur complement penalty dominating the single-step score in the high-censoring regime. With $d_m \approx 400$ and only $d_o \approx 384$ observed pixels, $\Sigma^{oo,c}$ is small and near-singular, making $(\Sigma^{oo,c})^+$ unreliable. The resulting $\log|\bar{\Sigma}^c|$ is numerically inaccurate and large in magnitude, overriding the observed-pixel likelihood term. Class 0 (a geometrically simple, symmetric digit) tends to have the smallest conditional log-determinant, so the penalty systematically favours it, causing the score collapse seen in the reconstructions.

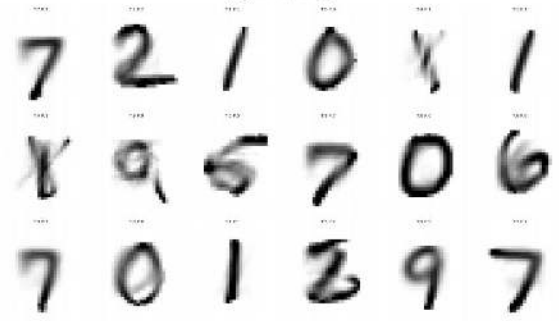
Question assigned to the following page: 3



(a) Censored (32%)



(b) Two-step

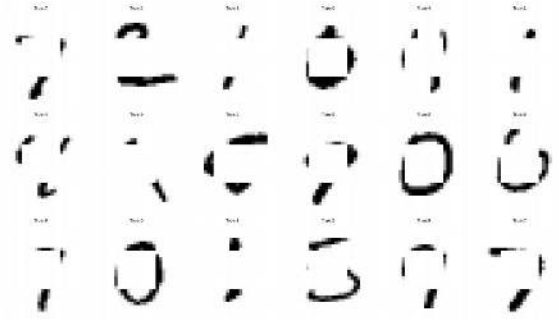


(c) Single-step

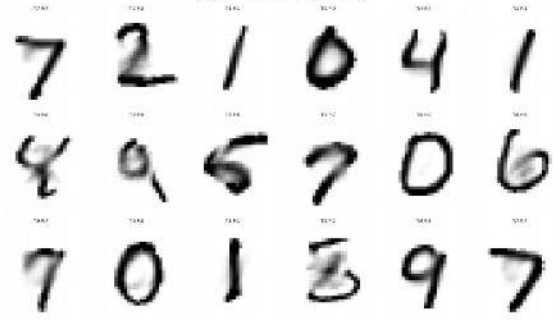
Figure 4: Reconstructions at 32% missing pixels (moderate-high censoring).

Moderate-high censoring (32%, Figure 4). With a wider observable border ring, both methods begin to recover meaningful digit structure. Two-step reconstructions are clear and largely correct. Single-step reconstructions are similar but still exhibit a small number of wrong predictions — a few images that two-step classifies correctly are still misclassified by single-step, producing fills with the wrong stroke pattern. The gap has narrowed substantially compared to 51%, consistent with the accuracy graph.

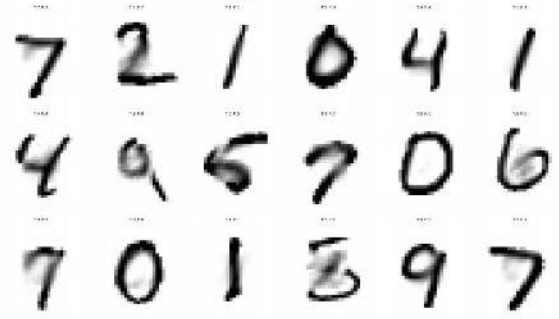
Question assigned to the following page: 3



(a) Censored (18%)



(b) Two-step

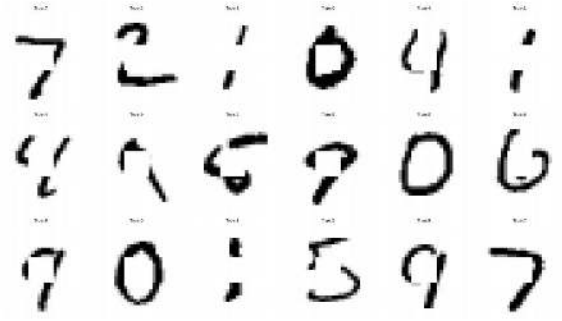


(c) Single-step

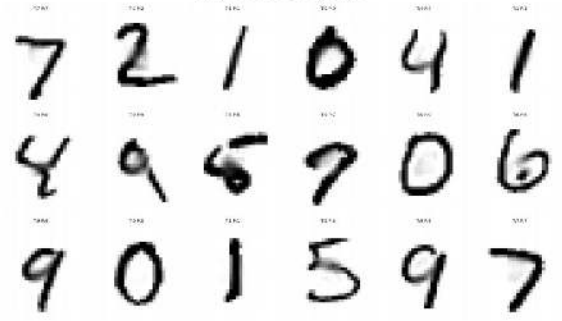
Figure 5: Reconstructions at 18% missing pixels (moderate, lecture default).

Moderate censoring (18%, Figure 5). At 18% missing pixels the two reconstruction grids are *visually identical* — same predicted labels, same fill patterns, same sharpness. This is consistent with the accuracy graph where both methods achieve nearly the same classification accuracy at this level. The class-conditional mean fills in the small missing central patch smoothly and in a way that is consistent with the observed outer strokes.

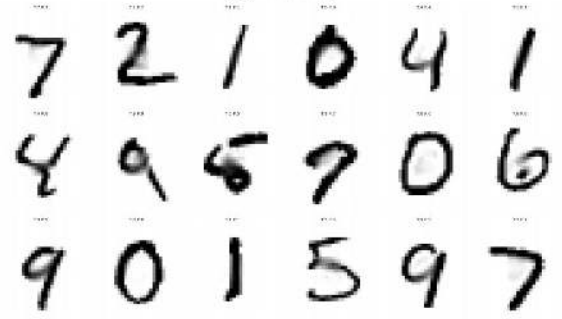
Question assigned to the following page: 3



(a) Censored (8%)



(b) Two-step

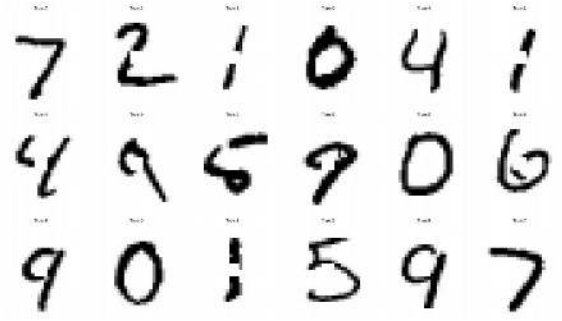


(c) Single-step

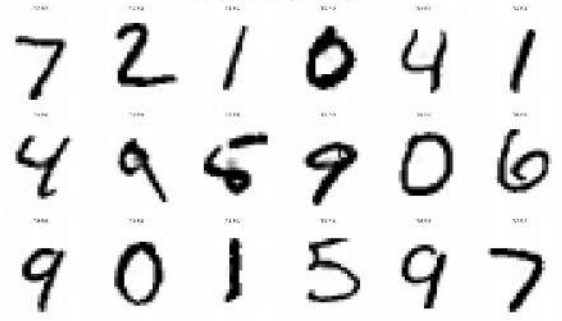
Figure 6: Reconstructions at 8% missing pixels (mild censoring).

Mild censoring (8%, Figure 6). The censored images themselves are already highly legible — the missing 8×8 patch removes only a small portion of most digit strokes. Both method's reconstructions are crisp, correct, and indistinguishable from each other. The class-conditional mean fills the tiny gap almost perfectly.

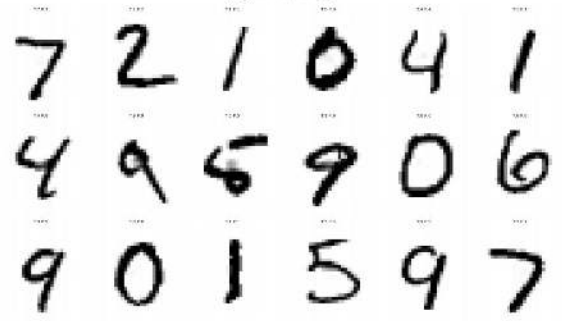
Question assigned to the following page: 3



(a) Censored (2%)



(b) Two-step



(c) Single-step

Figure 7: Reconstructions at 2% missing pixels (minimal censoring).

Minimal censoring (2%, Figure 7). At 2% censoring the censored images are visually identical to the originals — the 4×4 missing block is barely perceptible. Both methods reconstruct perfectly, and their outputs are indistinguishable from each other and from the uncensored images.

Overall. Reconstruction quality degrades monotonically as censoring increases, following classification accuracy trends exactly — as expected, since both methods share the identical fill formula (4) and differ only in which class label drives it. The most striking finding is at 51% censoring, where single-step collapses to predicting class 0 for almost all inputs due to numerical instability of the Schur complement penalty, producing clearly wrong reconstructions. Two-step, which ignores this unstable term entirely, is substantially more robust at high censoring. At 18% and below, both methods produce equivalent reconstructions and the distinction between them disappears entirely.

Question assigned to the following page: 3

References

1. Purushottam Kar. *Lecture 15: Generative Models*, CS771 Introduction to Machine Learning, IIT Kanpur, 2025–26.
2. Christopher M. Bishop. *Pattern Recognition and Machine Learning*, Springer, 2006. Section 2.3.
3. Hojjat Khodabakhsh. *Read MNIST Dataset*, Kaggle, 2020. `https://www.kaggle.com/code/hojjatk/read-mnist-dataset`